



INDIA.RUNS

Build what next India runs on

Team Name : RetiredAgent

Team Leader Name : Nabarup Ghosh

Problem Statement : Given 100,000 candidate profiles and a Senior AI Engineer job description, build an intelligent ranking system that identifies the top 100 most relevant candidates in under 5 minutes, filters out fabricated (honeypot) profiles, and provides explainable, factual reasoning for every recommendation.

Solution Overview

What is your proposed solution?

A hybrid "Teacher-Student" ranking engine that distills the judgment of a large language model into a lightweight XGBoost regression model for millisecond-level inference at scale. The system extracts 55 engineered features from raw candidate profiles, applies a multi-layered honeypot defense system, computes behavioral trust multipliers, and generates deterministic, fact-grounded reasoning strings for the final top 100 candidates. The entire pipeline processes 100,000 profiles end-to-end in roughly 2 minutes on commodity hardware.

What differentiates your approach from traditional candidate matching systems?

1. Traditional systems rely on keyword overlap (TF-IDF, BM25) or embedding similarity. Our system uses **LLM-distilled knowledge**: a *frontier LLM scores a stratified sample of 3,000 candidates offline*, and an *XGBoost model* learns those scoring patterns for instant deployment. This gives us **LLM-grade judgment at XGBoost speed**.
2. We **do not call any LLM at inference time**. *Zero API latency, zero hallucination risk, zero cost per query*.
3. Our **honeypot detector** uses **8 independent heuristic rules** (experience/education mismatch, salary inversion, time-travel logins, fake assessments) to identify and penalize fabricated profiles before they ever reach the scoring layer.
4. **Reasoning strings are generated from factual data fields** (years of experience, company names, response rates), not from LLM free-text generation, making every explanation auditable and grounded.

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

Dimensions	Key Signal	Weight
Role Fit	AI/ML Engineer title, not non-engineering roles	High
Core Skills	Information Retrieval, NLP, Ranking Systems, Search, Recommendation Engines	High
Experience Band	4 to 10 years (sweet spot: 6 to 8 years)	Medium
Company Type	Product companies preferred; consulting-only backgrounds penalized	Medium
Production ML	Evidence of shipping models to production (not just research)	High
Education	Graduate degree in CS/ML/Statistics preferred	Low
Behavioral	Active on platform, responsive to recruiters, reasonable notice period	Medium
Location	India-based candidates preferred	Low

Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

We go far deeper than keywords. Our 55-feature extraction pipeline captures:

- **Semantic role classification:** We classify titles into AI, Engineering, and Non-Engineering buckets using curated keyword dictionaries, not just string matching.
- **Career trajectory analysis:** We compute years of experience from career history, detect product vs. consulting backgrounds, and flag career gaps.
- **Skill-to-JD alignment scoring:** We match candidate skills against retrieval/NLP/ranking keyword sets and compute weighted overlap ratios.
- **Behavioral profiling:** Recruiter response rates, platform login recency, notice period length, and assessment completion rates are all factored in as trust and availability signals.
- **Anomaly detection:** Impossible combinations (e.g., 15 years experience with a 2023 graduation date) are flagged as fabrication indicators.

Ranking Methodology

1. How does your system retrieve, score, and rank candidates?

The system uses a 3-stage scoring pipeline:

A. Technical Score (XGBoost): A regression model trained on LLM-labeled data predicts a 0-to-5 relevance score from 52 engineered features. This is the primary ranking signal.

B. Behavioral Multiplier: A 0-to-1 modifier based on recruiter response rate, login recency, notice period, and assessment completion. Penalizes "ghost" candidates who look great on paper but are unresponsive.

C. Honeypot Penalty: A binary gate that zeros out candidates flagged as fabricated, plus a graduated suspicion penalty for borderline profiles.

Final Score = XGBoost Score x Behavioral Multiplier x Honeypot Penalty

2. What models, algorithms, or heuristics are used?

A. XGBoost Regressor (300 trees, depth 6, learning rate 0.05) trained on 1,311 LLM-labeled samples with 5-fold cross-validation (CV RMSE: 0.46)

B. 8 rule-based honeypot heuristics covering experience/education mismatch, career/experience mismatch, title/description mismatch, salary inversion, time-travel logins, and fake assessment detection

C. Weighted behavioral scoring formula combining 4 behavioral dimensions with tuned coefficients

3. How are multiple candidate signals combined into a final ranking?

The three scores are combined multiplicatively, not additively. This is intentional: a candidate who is technically strong (**high XGBoost score**) but behaviorally dead (**low behavioral multiplier**) or flagged as fake (**honeypot penalty of 0**) gets correctly suppressed. Ties in the final score are broken by candidate ID in **ascending** order, as required by the submission specification.

Explainability & Data Validation

1. How are ranking decisions explained?

Every top-100 candidate receives a dynamically generated reasoning string built from their actual profile data. The reasoning generator follows a deterministic template that assembles facts:

- *Current title and years of experience*
- *Whether they have product company experience (and which company)*
- *Whether they have relevant retrieval/ranking/NLP skills*
- *Their recruiter response rate percentage*
- *Their notice period*
- *Any notable gaps (e.g., "120-day notice period exceeds preferred 30 days")*

Example output: "Staff Machine Learning Engineer with 8.6 years experience at Yellow.ai; career demonstrates hands-on ranking/retrieval system deployment; 83% recruiter response rate."

2. How do you prevent hallucinations or unsupported justifications?

We never call an LLM at inference time. Every word in the reasoning string is derived from a specific data field in the candidate's profile. The reasoning generator is a pure Python function with zero randomness. Given the same input, it always produces the same output. There is no generative model involved at this stage, so hallucination is architecturally impossible.

3. How does your solution handle inconsistent, low-quality, or suspicious profiles?

Our 8-rule honeypot detector identifies fabricated profiles through:

1. *Experience years that contradict graduation dates*
2. *Career history duration that contradicts stated experience*
3. *Job titles that contradict career descriptions*
4. *Salary expectations inverted (minimum > maximum)*
5. *Last login timestamps that precede account creation (time travel)*
6. *Perfect assessment scores across unrelated domains (fake assessments)*

Candidates violating 3 or more rules are hard-flagged. All others receive a graduated suspicion score that softly penalizes their final rank.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

Offline Phase (One-time, takes roughly 30 minutes):

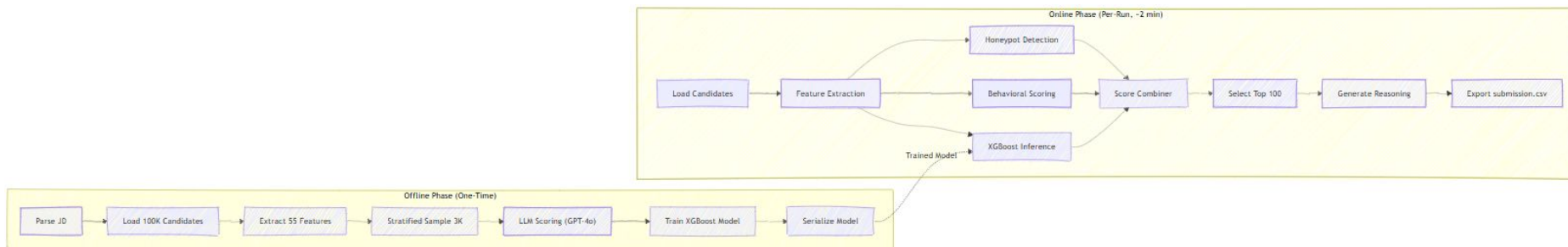
1. Parse the JD to extract key requirements, skill keywords, and disqualifiers.
2. Load 100,000 candidates and extract 55 features.
3. Stratified-sample 3,000 candidates across 6 buckets (clearly relevant, plausibly relevant, edge cases, clearly irrelevant, honeypots, random).
4. Query an LLM (GPT-4o / Claude / Gemini) to score each sampled candidate on a 0-to-5 scale.
5. Train an XGBoost regression model on the LLM labels using 5-fold cross-validation.

Serialize the trained model and feature configuration.

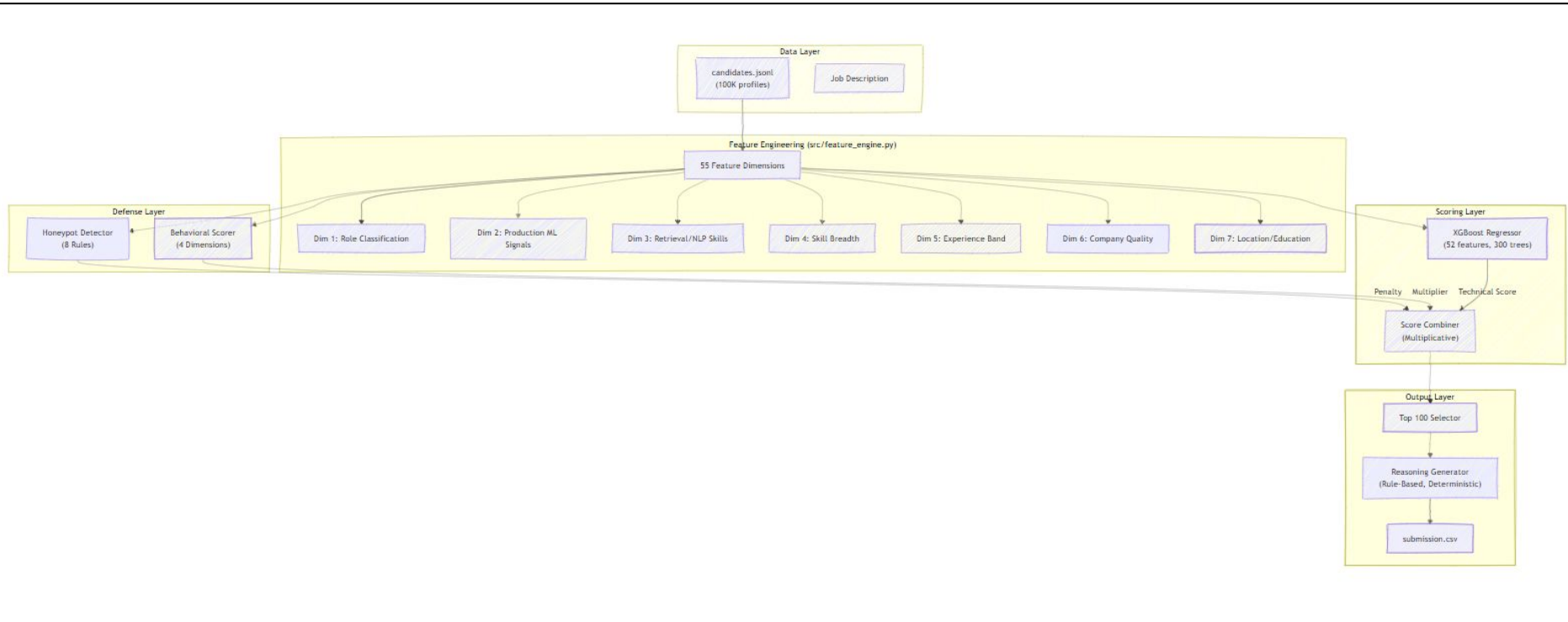
Online Phase (Per-run, takes roughly 2 minutes):

1. Load all 100,000 candidates from JSONL.
2. Extract 55 features using the same pipeline.
3. Run the honeypot detector (8 rules) to flag fabricated profiles.
4. Compute behavioral trust multipliers.
5. Run XGBoost inference on all 100,000 candidates.
6. Combine scores multiplicatively and select the top 100.
7. Generate deterministic reasoning strings for the top 100.
8. Export the final submission.csv with candidate_id, rank, score, and reasoning.

Mermaid Diagram (End-to-End Workflow)



System Architecture



Results & Performance

What results or insights demonstrate ranking quality?

Metric	Value
XGBoost CV RMSE	AI/ML Engineer title, not non-engineering roles
Honeypots in Top 100	0 out of 355 detected
Top 10 Title Relevance	100% AI/ML engineering roles
Score Range (Top 100)	0.6479 to 0.7477
Unique Reasoning Strings	100 out of 100 (zero duplicates)
Official Validator	All checks passed

Top 3 Candidates Produced:

Rank	Title	Company	Score
1	Junior ML Engineer	Rephrase.ai	0.7477
2	Data Scientist	Jio (Product Co.)	0.7318
3	Data Scientist	Verloop.io	0.7318

Key Insight: The model successfully learned to rank candidates from AI-native product companies (Rephrase.ai, Jio (Product Co., Verloop.io) above those from IT services firms, reflecting the JD's explicit preference for product company experience.

How does your solution meet the challenge's runtime and compute constraints?

Phase	Time
Data Loading	~35s
Feature Extraction	~60s
Honeypot Detection	~6s
Behavioral Scoring	<1s
XGBoost Inference (100K rows)	~6s
Reasoning Generation (100 rows)	<1s
Total Pipeline	~130s (well under 300s limit)

No GPU required. Runs on any machine with Python 3.9+ and 4GB RAM

Technologies Used

Technology	Purpose	Why Selected
Python 3.9+	Core language	Industry standard for ML pipelines
XGBoost	Candidate scoring model	Best-in-class tabular ML; fast inference, handles missing values natively
Pandas	Data manipulation	Efficient columnar operations on 100K rows
scikit-learn	Cross-validation, metrics	Reliable ML evaluation toolkit
GPT-4o (OpenAI)	Offline training label generation	Frontier LLM for high-quality relevance judgments
Joblib	Model serialization	Fast pickle for sklearn-compatible models

Why this stack?

We deliberately **avoided heavyweight frameworks** (LangChain, vector databases, embedding models) because the problem does not require semantic search over unstructured text. The candidates are structured *JSON records with well-defined fields*. **XGBoost on engineered features is the optimal tool for structured tabular ranking**. It gives us sub-second inference on 100,000 rows, which no embedding-based approach can match.

Submission Assets

GitHub Link: <https://github.com/Nabarup1/redrob-candidate-ranker>

Submission File: [output/submission.csv \(100 ranked candidates with scores and reasoning\)](#)



INDIA.RUNS

Build what next India runs on

THANK YOU